Chapter 01: Getting Started with LLMs and RAG

Project: LLM & RAG. Goal of this chapter: build a working retrieval-augmented generation pipeline twice, first from scratch and then with LlamaIndex and ChromaDB, so you understand every moving part before a framework hides it.

1. What you will build

By the end of this chapter you will have:

- called an LLM directly through Groq and seen exactly where it fails on private data,
- built a tiny RAG pipeline in about 60 lines of plain Python (chunking, embeddings, cosine retrieval, prompt assembly),
- rebuilt the same pipeline with LlamaIndex and a persistent ChromaDB store,
- learned which knobs matter (chunk size, overlap, top_k, temperature) and why.

The stack matches what you would use for a chatbot service later: Groq for fast inference, a local embedding model, ChromaDB for vectors, LlamaIndex for orchestration.

2. Why RAG exists

An LLM is a compressed statistical picture of its training data. That gives it four limits that no prompt can fully remove:

1. **Knowledge cutoff.** It knows nothing after training.
2. **No private data.** It has never seen your lab notes, papers or company documents.
3. **Hallucination.** When it lacks the fact, it still produces fluent text, because fluency is what it was trained for.
4. **Finite context.** You cannot paste an entire corpus into every prompt, and cost and latency grow with prompt length.

RAG addresses all four with one idea: **do not ask the model to remember; ask it to read.** At question time, retrieve the few passages that matter and place them in the prompt. The model becomes a reasoning and writing engine over evidence you supplied, and you can show the user the sources.

Why not fine-tune instead? Fine-tuning changes behaviour and style well, but it is a poor way to inject facts: it is expensive, facts get blurred, you cannot cite sources, and updating one fact means retraining. RAG updates by re-indexing a document.

3. The mental model

Every RAG system, however elaborate, is two pipelines:

Indexing (offline, once per document change)

load documents, split into chunks, embed each chunk into a vector, store vectors with the text and metadata.

Querying (online, per question)

embed the question, find the nearest chunk vectors, build a prompt from question plus chunks, generate the answer, return the answer with its sources.

The key concept is the **embedding**: a model maps text to a point in a high-dimensional space where texts with similar meaning land close together. Retrieval is then just nearest-neighbour search. We measure closeness with **cosine similarity**, which compares direction rather than length, so a long and a short passage about the same topic still match.

4. Setup

Use Python 3.10 or newer and a fresh virtual environment.

```
pip install groq sentence-transformers numpy chromadb llama-index llama-index-llms-groq llama-index-embeddings-huggingface llama-index-vector-stores-chroma
```

Create a free key in the Groq console and export it:

```
export GROQ_API_KEY='your-key-here'
```

Model names on hosted platforms change over time. This chapter uses `llama-3.3-70b-versatile`; if Groq has retired it, pick a current chat model from their model list and change one constant.

5. Step 1: Ask the LLM directly (and watch it fail)

We invent a document the model cannot possibly have seen. That is the point: it lets us tell retrieval working from the model guessing.

```
# step1_direct.py
from groq import Groq

MODEL = 'llama-3.3-70b-versatile'
client = Groq()  # reads GROQ_API_KEY from the environment

def ask(messages, temperature=0.0):
    resp = client.chat.completions.create(
        model=MODEL, messages=messages, temperature=temperature
    )
    return resp.choices[0].message.content

question = 'How many days of remote work per week does Helix Robotics allow
for new hires?'
print(ask([{'role': 'user', 'content': question}]))
```

Why `temperature=0`? Temperature scales how randomly the model samples the next token. For question answering over documents you want the most probable, repeatable answer, not creativity. It also makes your experiments comparable between runs.

Expect a confident, generic answer or a polite refusal. Either way it is not grounded in anything real. That gap is what RAG fills.

6. Step 2: Embeddings and similarity in five minutes

```
# step2_embeddings.py
import numpy as np
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('BAAI/bge-small-en-v1.5')

texts = [
    'New employees may work remotely two days per week.',
    'The cafeteria serves lunch from noon to 2 pm.',
    'Remote work policy for recently hired staff.',
]
vecs = embedder.encode(texts, normalize_embeddings=True)
print(vecs.shape)  # (3, 384)

# With normalized vectors, dot product == cosine similarity
print(vecs @ vecs.T)
```

Texts 0 and 2 share almost no words in common beyond the topic, yet they score far higher with each other than with text 1. That is semantic search, and it is the reason embeddings beat keyword matching for questions phrased differently from the source.

Why `normalize_embeddings=True`? Normalising every vector to length 1 turns cosine similarity into a plain dot product, which is faster and lets vector databases use simple inner-product indexes. Many embedding models are trained assuming this.

Why a small model? `bge-small-en-v1.5` is 384-dimensional, runs on CPU and is accurate enough for learning. Larger embedders improve recall on hard corpora, at the cost of speed and index size. Whichever you choose, you must embed documents and queries with the **same** model, because vectors from different models live in unrelated spaces.

7. Step 3: RAG from scratch

First, some private knowledge. Save this as `data/handbook.txt`:

```
Helix Robotics Employee Handbook

Remote work: New hires may work remotely up to two days per week during their
first six months. After six months, managers may approve up to three remote
days per week.

Equipment: Every employee receives a laptop and one external monitor.
Additional peripherals require written approval from IT.

Leave: Full-time employees accrue 1.5 days of paid leave per month. Unused
leave up to 10 days carries over to the next calendar year.

Security: All laptops must use full-disk encryption. Lost devices must be
reported to the security team within 24 hours.
```

Now the whole pipeline in one file:

```
# step3_scratch_rag.py
import numpy as np
from groq import Groq
from sentence_transformers import SentenceTransformer

MODEL = 'llama-3.3-70b-versatile'
llm = Groq()
embedder = SentenceTransformer('BAAI/bge-small-en-v1.5')

# ---------- 1. Load ----------
text = open('data/handbook.txt', encoding='utf-8').read()

# ---------- 2. Chunk ----------
def chunk_words(text, size=60, overlap=15):
    words = text.split()
    step = size - overlap
    chunks = []
    for start in range(0, len(words), step):
        piece = words[start:start + size]
        chunks.append(' '.join(piece))
        if start + size >= len(words):
            break
    return chunks

chunks = chunk_words(text)
```

```
# ---------- 3. Embed and store (an in-memory 'vector DB') ----------
chunk_vecs = embedder.encode(chunks, normalize_embeddings=True)

# ---------- 4. Retrieve ----------
def retrieve(question, k=3):
    q = embedder.encode([question], normalize_embeddings=True)[0]
    scores = chunk_vecs @ q
    top = np.argsort(scores)[::-1][:k]
    return [(float(scores[i]), chunks[i]) for i in top]

# ---------- 5. Augment and generate ----------
def answer(question, k=3):
    hits = retrieve(question, k)
    context = '\n\n'.join(f'[{i+1}] {c}' for i, (_, c) in enumerate(hits))
    system = (
        'Answer using ONLY the context. Cite sources like [1]. '
        'If the context does not contain the answer, say you do not know.'
    )
    user = f'Context:\n{context}\n\nQuestion: {question}'
    resp = llm.chat.completions.create(
        model=MODEL, temperature=0,
        messages=[{'role': 'system', 'content': system},
                  {'role': 'user', 'content': user}],
    )
    return resp.choices[0].message.content, hits

if __name__ == '__main__':
    q = 'How many remote days can new hires have per week?'
    ans, hits = answer(q)
    print(ans)
    for score, c in hits:
        print(round(score, 3), c[:80], '...')
```

Why each design decision was made

Chunking. Embedding a whole document produces one blurry vector that matches everything a little and nothing well. Chunks make each vector about one idea. But chunks that are too small lose context (a sentence saying 'it' with no antecedent), and chunks that are too large dilute the vector and waste prompt space. A common starting range is 256 to 512 tokens.

Overlap. Without overlap, a fact that straddles a boundary is split across two chunks and neither retrieves well. A 10 to 20 percent overlap is cheap insurance. The cost is duplicated storage and occasionally duplicated passages in the top results.

top_k. Too low and you miss the passage that holds the answer; too high and you flood the prompt with distractors, raising cost and sometimes lowering answer quality (models can be pulled off course by irrelevant text). Start at 3 to 5 and tune against real questions.

The grounding instruction. 'Answer using ONLY the context' and 'say you do not know' is what converts retrieval into trustworthiness. Without it the model blends its own memory with your documents, and you cannot tell which parts are sourced.

Numbered sources. Labelling chunks `[1]`, `[2]` costs nothing and gives you citations, which is the main way users and you verify an answer.

Now run the question from Step 1 again. Compare the two answers: the grounded one should state two days for new hires, citing the handbook chunk.

8. Step 4: The same pipeline with LlamaIndex and ChromaDB

Frameworks earn their keep when you need persistence, many file types and swappable components. Everything you wrote by hand now maps to a named piece:

From scratch	LlamaIndex / Chroma
`open(...).read()`	`SimpleDirectoryReader`
`chunk_words`	`SentenceSplitter` (sentence-aware, token-based)
`embedder.encode`	`HuggingFaceEmbedding`
numpy array	`ChromaVectorStore` over a persistent collection
`retrieve` + prompt + LLM call	`as_query_engine`

```
# step4_llamaindex.py
import chromadb
from llama_index.core import (
    Settings, SimpleDirectoryReader, StorageContext, VectorStoreIndex,
)
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
```

```
from llama_index.llms.groq import Groq
from llama_index.vector_stores.chroma import ChromaVectorStore

# Global configuration: every component reads from Settings
Settings.llm = Groq(model='llama-3.3-70b-versatile', temperature=0.0)
Settings.embed_model = HuggingFaceEmbedding(model_name='BAAI/bge-small-en-
v1.5')
Settings.node_parser = SentenceSplitter(chunk_size=256, chunk_overlap=32)

# Persistent vector store on disk
client = chromadb.PersistentClient(path='./chroma_db')
collection = client.get_or_create_collection('ch01_handbook')
vector_store = ChromaVectorStore(chroma_collection=collection)

if collection.count() == 0:
    # First run: index the documents
    docs = SimpleDirectoryReader('data').load_data()
    storage = StorageContext.from_defaults(vector_store=vector_store)
    index = VectorStoreIndex.from_documents(docs, storage_context=storage)
else:
    # Later runs: reuse stored vectors, no re-embedding
    index = VectorStoreIndex.from_vector_store(vector_store)

query_engine = index.as_query_engine(similarity_top_k=3)
response = query_engine.query('How many remote days can new hires have per
week?')

print(response)
for node in response.source_nodes:
    print(round(node.score, 3), node.metadata.get('file_name'),
node.text[:80])
```

Why `Settings`? It sets the LLM, embedder and splitter once, so no component silently falls back to a default (LlamaIndex defaults to OpenAI, which would fail without that key and, worse, would use a different embedding model than you intended).

Why persist and branch on `collection.count()`? Embedding is the slow, repeatable part of indexing. Persisting means you pay it once; reloading with `from_vector_store` skips it. The branch is a simple guard so re-running the script does not insert duplicate chunks. For real systems you will track document hashes and update only changed files.

Why print `source_nodes`? The scores and file names are your debugging window. When an answer is wrong, the first question is always: was the right chunk retrieved? If not, fix chunking or retrieval. If yes, fix the prompt or model. Mixing those two up is the most common time sink in RAG work.

9. Experiments to run

Change one thing at a time and watch the retrieved chunks, not only the final answer.

1. Set `chunk_size` to 32, then 1024. Which questions break in each case?
2. Set `top_k` to 1, then 10. When does the answer get worse with more context?
3. Ask something the handbook does not cover ('What is the dress code?'). Does the model say it does not know? Remove the grounding instruction and ask again.
4. Paraphrase questions heavily ('How often can someone just hired stay home?'). Does retrieval still find the right chunk?
5. Swap the embedder for `BAAI/bge-base-en-v1.5` and compare scores.

10. Common failure modes

- **Right chunk never retrieved.** Chunking split the fact, or the question and document use different vocabulary and the embedder is weak for the domain.
- **Right chunk retrieved, wrong answer.** Prompt is vague, `top_k` brings in conflicting passages, or the answer needs reasoning across several chunks.
- **Confident answers with no support.** Missing 'say you do not know' instruction, or the model is using its own memory.
- **Duplicate results.** Re-running ingestion without a guard, or too much overlap.
- **Different embedding models at index and query time.** Similarity scores look random.

11. Chapter summary

- RAG replaces 'remember' with 'read': retrieve evidence, then generate from it.
- The pipeline is chunk, embed, store, retrieve, augment, generate. You built each step by hand before using a framework.
- Quality is decided mostly by chunking and retrieval, not by the LLM. Always inspect retrieved chunks first.

- Ground the model with explicit instructions and numbered sources so answers are checkable.

12. Where Chapter 02 goes next

This chapter used the simplest retrieval: one embedding search, top_k results. Chapter 02 improves retrieval quality: better chunking strategies, metadata filtering, hybrid keyword plus vector search, reranking, and a small evaluation set so you can measure improvements instead of guessing.